Reporte

Nombre: Valentin Monsivais Martinez	**Matrícula**: 2157656
Nombre del curso: Ingenieria en Tecnologia de Software	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: Resumen
Fecha: 26/10/2026	
Bibliografia (Formato APA)	

Objetivo:

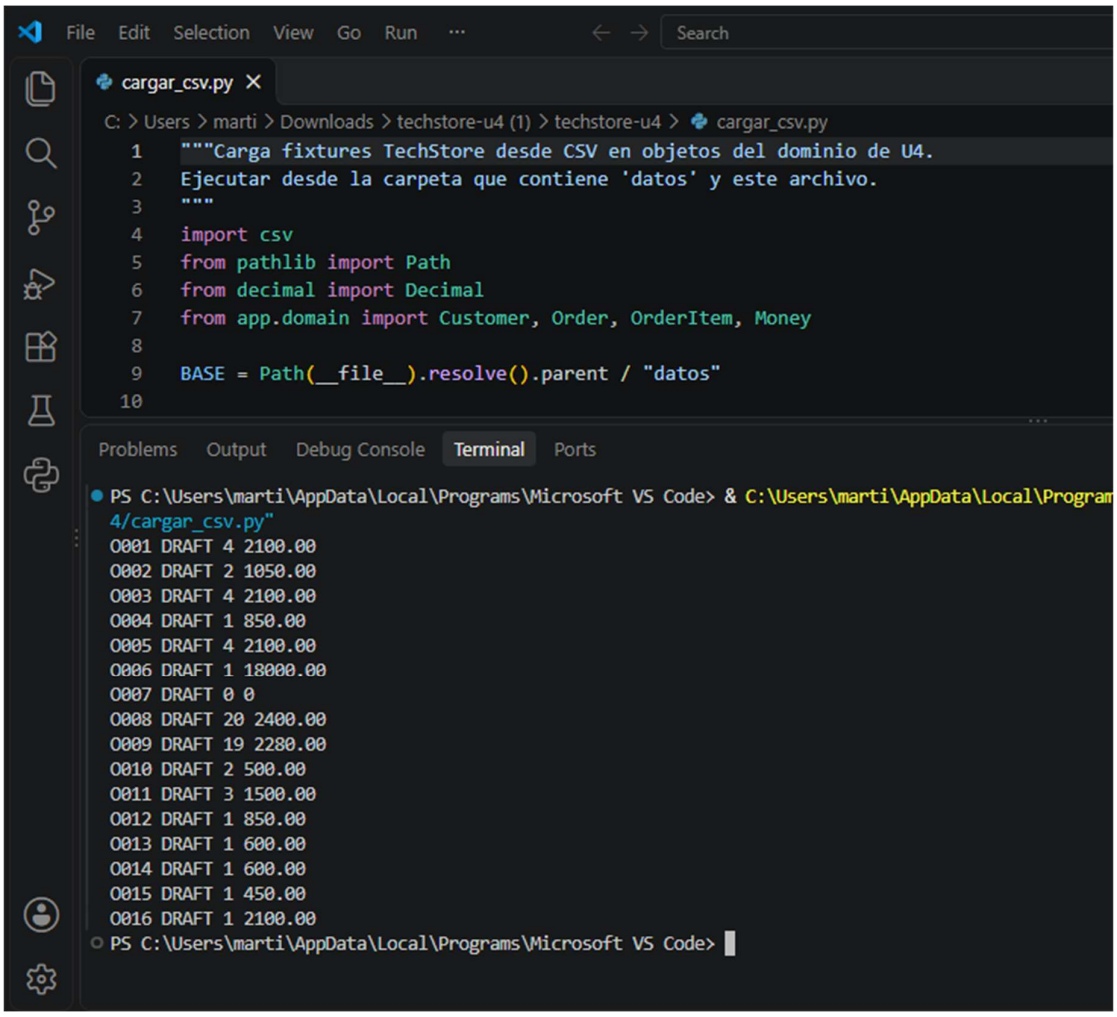
Implementar en Python el núcleo de negocio de un sistema de pedidos de modo que las reglas del negocio queden protegidas en el dominio. La implementación se comprueba con 15 escenarios y pruebas unitarias en pytest.

Reporte

Procedimiento:

Pruebas unitarias

Captura de carga exitosa de los 16 pedidos.



Reporte

Bitácora resuelta de los 15 escenarios E01–E15.

BITACORA.md

Archivo Editar Ver

Bitácora E01–E15 · TechStore U4

Resultados reales obtenidos con `python ejecutar_escenarios.py` (15 de 15 coinciden con lo esperado).

Caso	Estado inicial	Operación	Resultado real	Evento emitido	Explicación	Prueba (tests/)
E01 (O001)	DRAFT	evaluar descuento	10% → total 2100.00 → final 1890.00	Ninguno (es una consulta)	Cumple la regla del 10%: cliente VIP, total > 500, 4 unidades (≥ 4) y sin promoción.	`test_descuento_vip_10_por_ciento`
E02 (O002)	DRAFT	evaluar descuento	0% → total 1050.00 → final 1050.00	Ninguno (es una consulta)	No cumple el 10% (solo 2 unidades, se exigen ≥ 4) ni el 5% (total no supera 5000).	`test_vip_necesita_4_unidades`
E03 (O003)	DRAFT	confirmar	RECHAZO: InvalidOrderData (estado DRAFT)	Ninguno (falló `confirmar`)	El pedido tiene artículos (4 unidades), así que la precondición que falla es el cliente activo. `confirm()` valida antes de cambiar el estado: sigue en DRAFT.	`test_confirmar_cliente_inactivo_rechazado; test_confirm_cliente_inactivo_422`
E04 (O005)	DRAFT	evaluar descuento	0% → total 2100.00 → final 2100.00	Ninguno (es una consulta)	Tiene promoción activa; `~ActivePromotionSpec` excluye el 10% aunque haya 4 unidades.	`test_promocion_activa_bloquea_ambos_descuentos`

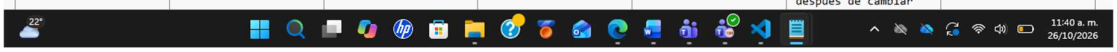
BITACORA.md

Archivo Editar Ver

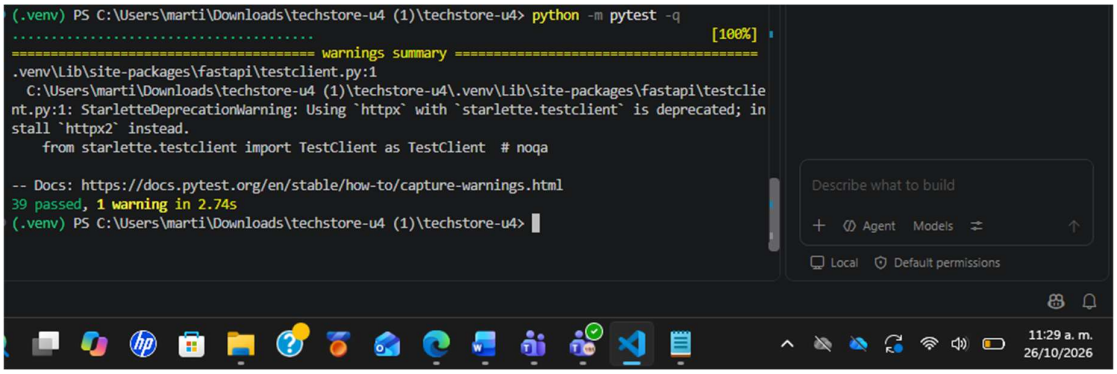
E05 (O006)	DRAFT	evaluar descuento	5% → total 18000.00 → final 17100.00	Ninguno (es una consulta)	Solo 1 unidad, así que no llega al 10%; el total supera 5000 y no hay promoción, aplica el 5%.	`test_limite_5000_es_estricto`
E06 (O007)	DRAFT	confirmar	RECHAZO: InvalidOrderData (estado DRAFT)	Ninguno (falló `confirmar`)	Pedido vacío (0 unidades); `confirm()` exige al menos un artículo. Estado sin cambios.	`test_confirmar_pedido_vacio_rechazado; test_confirm_pedido_vacio_422`
E07 (O008)	DRAFT	confirmar	OK, termina en CONFIRMED	OrderConfirmed	Valor frontera: 20 unidades es exactamente el máximo permitido. Hay artículos, cliente activo y estado DRAFT, así que la transición es válida y se registra el evento.	`test_20_unidades_exactas_permitidas`
E08 (O009)	DRAFT	agregar SKU003 x2	RECHAZO: OrderItemLimitExceeded (estado DRAFT)	Ninguno (falló `agregar SKU003 x2`)	Ya hay 19 unidades; 19 + 2 = 21 supera el límite de 20. `add_item()` falla antes de modificar `items`.	`test_21_unidades_rechazadas_y_pedido_intacto`
E09 (O010)	DRAFT	evaluar descuento	0% → total 500.00 → final 500.00	Ninguno (es una consulta)	Valor frontera: el total es exactamente 500 y `MinimumAmountSpec` usa `>`, no `>=`. No hay descuento.	`test_limite_500_es_estricto`

Reporte

E09 (O010)	DRAFT	evaluar descuento	0% → total 500.00 → final 500.00	Ninguno (es una consulta)	Valor frontera: el total es exactamente 500 y `MinimumAmountSpec` usa `>`, no `>=`. No hay descuento.	`test_limite_500_es_estricto`
E10 (O011)	DRAFT	evaluar descuento	0% → total 1500.00 → final 1500.00	Ninguno (es una consulta)	Solo 3 unidades (se exigen ≥ 4 para el 10%) y el total no supera 5000 para el 5%.	`test_vip_necesita_4_unidades`
E11 (O012)	DRAFT	confirmar → enviar sin guía	RECHAZO en `enviar sin guía`: InvalidOrderData (estado CONFIRMED)	OrderConfirmed; ninguno en `ship`	`confirm()` es válido y emite su evento. `ship()` exige guía no vacía; sin ella falla, el pedido sigue en CONFIRMED y no se emite OrderShipped.	`test_enviar_sin_guia_rechazado; test_ship_guia_vacia_422`
E12 (O013)	DRAFT	confirmar → confirmar (2.ª vez)	RECHAZO en la 2.ª vez: InvalidOrderTransition (estado CONFIRMED)	OrderConfirmed (solo uno); ninguno en la 2.ª	`confirm()` solo se permite desde DRAFT. La segunda llamada lanza la excepción y no agrega un segundo evento.	`test_confirmacion_doble_rechazada; test_confirm_dos_veces_409`
E13 (O014)	DRAFT	confirmar → enviar → cancelar	RECHAZO en `cancelar`: InvalidOrderTransition (estado SHIPPED)	OrderConfirmed, OrderShipped; ninguno en `cancel`	`cancel()` exige estado CONFIRMED; un pedido ya enviado no se puede cancelar. Sigue en SHIPPED.	`test_cancelar_desde_shipped_rechazado; test_cancel_desde_shipped_409`
E14 (O015)	DRAFT	confirmar → enviar → devolver	OK, termina en RETURNED	OrderConfirmed, OrderShipped, OrderReturned	Cadena válida: `return_order()` exige SHIPPED, y cada transición registra su evento después de cambiar	`test_flujo_entrega_y_devolucion`
E15 (O016)	DRAFT	confirmar → enviar → entregar	OK, termina en DELIVERED	OrderConfirmed, OrderShipped, OrderDelivered	Cadena válida: `deliver()` exige SHIPPED y registra OrderDelivered.	`test_flujo_entrega_y_devolucion`



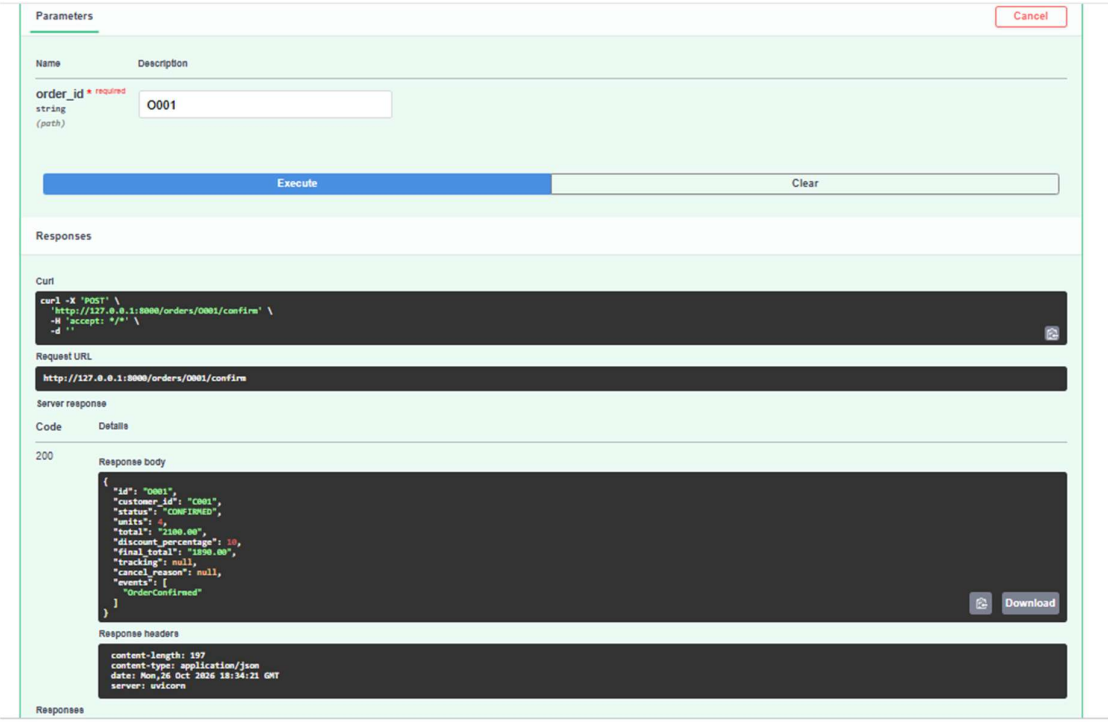
Pruebas pytest y captura de resultados.



Reporte

Capturas de Swagger /docs mostrando una operación exitosa y una rechazada.

Exitosa



Rechazada

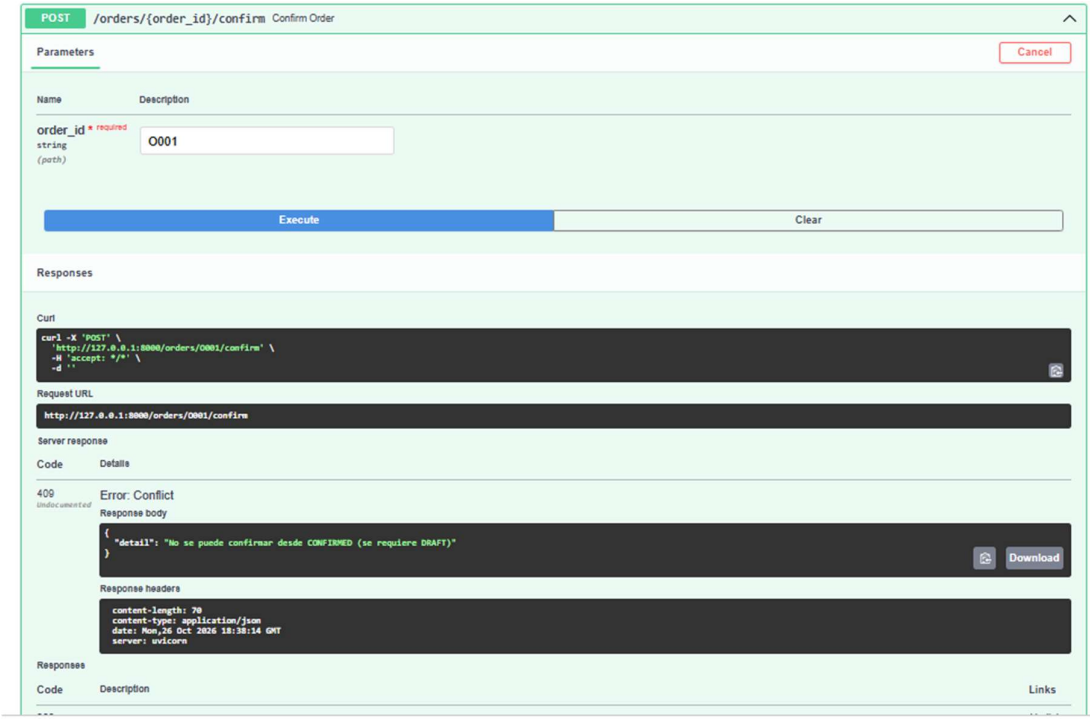
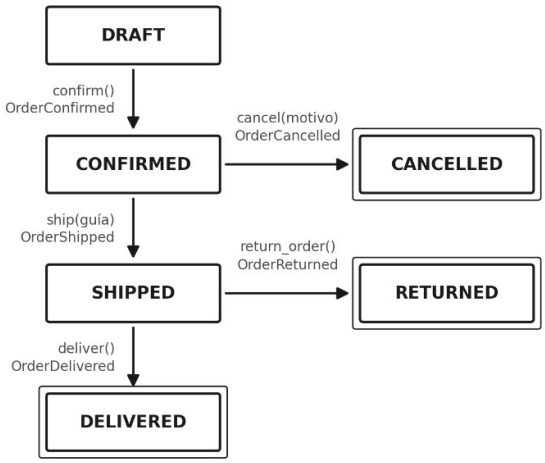


Diagrama de estados y explicación de los Domain Events.



Estado inicial: DRAFT. Borde doble = estado final.

Cada flecha: validar → cambiar estado → registrar evento.

Un Domain Event es un objeto que representa un hecho de negocio que ya ocurrió. Por eso sus nombres van en pasado (OrderConfirmed, no ConfirmOrder). En TechStore se implementa como una clase DomainEvent inmutable (frozen=True) con tres datos: el nombre del evento, el order_id del pedido y la hora en que ocurrió (occurred_at) en UTC. Al ser inmutable, un evento no se puede alterar después de creado, igual que un hecho pasado no se puede cambiar.
El Order (raíz del agregado) acumula sus eventos en una lista interna y los expone solo para lectura. Cada transición válida registra exactamente un evento:

Operación	Transicion	Evento Registrado
confirm()	DRAFT - CONFIRMED	OrderConfirmed
ship(guía)	CONFIRMED - SHIPPED	OrderShipped
cancel(motivo)	CONFIRMED - CANCELLED	OrderCancelled
deliver()	SHIPPED - DELIVERED	OrderDelivered
return_order()	SHIPPED - RETURNED	OrderReturned

El orden dentro de cada método es siempre el mismo: primero se valida, luego se cambia el estado y hasta el final se registra el evento. Debido a esto, una operación invalida lanza una excepción antes de tocar el estado e impide el evento. Esto se puede observar en los escenarios de la sección anterior. En el E12 al confirmar el evento dos veces, la segunda llamada falla y el pedido queda únicamente como un “OrderConfirmed”. Por otro lado en el E13, tratar de cancelar un pedido enviado, la cancelación se rechaza y solo existen “OrdenConfirmed” y “OrderSipped”. Estos eventos permiten que otros eventos, como un evento de envio o el control de inventado, puedan actuar de acuerdo a OrderConfirmed sin que Order conozca su existencia.

Reflexión breve: tres errores encontrados, cómo los solucionaste y qué aprendiste.

Error 1: pytest no se ejecutaba

Al intentar correr las pruebas, pytest no arrancaba y no se obtenían resultados.

Investigué la causa y vi que el paquete no estaba instalado en el entorno virtual del proyecto. Lo instalé con pip install pytest (junto con las demás dependencias: fastapi, uvicorn y httpx) y, al volver a ejecutar python -m pytest -q, las pruebas corrieron correctamente.

Error 2: Swagger aparecía en blanco

Al abrir el enlace que mostraba la terminal después de ejecutar python -m uvicorn app.api:app --reload, la ventana aparecía sin contenido.

Después de probar varias opciones, me di cuenta de que ese enlace apunta a la raíz del servidor, donde la API no tiene ninguna página. La documentación interactiva de FastAPI se encuentra en la ruta /docs, así que la dirección correcta es http://127.0.0.1:8000/docs.

Error 3: error al consultar el pedido O001 en Swagger

Al probar el endpoint con el pedido O001, Swagger devolvía un error de archivo no encontrado.

La causa fue que había levantado el servidor desde una terminal que no estaba en la carpeta raíz del proyecto, por lo que la API no encontraba el código ni la carpeta datos/ con los CSV. [Solo si lo confirmas: “Lo solucioné abriendo una terminal nueva en la carpeta del proyecto, con el entorno virtual activo, y levantando de nuevo la API; después de eso el pedido O001 respondió correctamente.”]

Aprendizaje

Estos errores me permitieron comprender que no basta con simplemente tener un código correcto, también es importante conocer como ejecutarlo y el entorno en el que se ejecuta. Ver el sistema funcionar a tiempo real con las pruebas, el Api y la documentación permiten conocer como se conecta el dominio, la API y las pruebas en general.

Preguntas

1. **¿Por qué se usa Decimal en lugar de float para precios?**

float guarda los números en binario y no puede representar exactamente valores como 0.1, así que al operar se acumulan pequeños errores. Decimal trabaja en base 10 y da resultados exactos. Por eso Money usa Decimal y rechaza cualquier float desde su constructor.

2. **¿Cuál es la diferencia entre contar renglones y sumar unidades?**

Un renglón es una línea del pedido (un producto con su cantidad), y las unidades son la suma de todas esas cantidades. Si el límite contara renglones, ese mismo pedido tendría solo 2 y se aceptaría, lo cual violaría la regla.

3. **¿Qué ventaja tiene encapsular una regla en Specification?**

Cada condición del negocio (cliente VIP, monto mínimo, unidades mínimas, promoción activa) vive en su propia clase pequeña, con un nombre claro y un solo método, is_satisfied_by. Si una regla cambia, por ejemplo el umbral de 500, se modifica en un solo lugar. Además, cada condición se puede probar por separado.

4. **¿Por qué los eventos se registran solo después de una transición válida?**

Un Domain Event comunica un hecho que **ya ocurrió**, como OrderConfirmed. Si se registrara antes de validar, una operación rechazada dejaría un evento falso, y otros

componentes podrían reaccionar a algo que nunca pasó. Por eso el orden en cada método es validar, cambiar el estado y registrar el evento.

5. **¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?**

Casi todo. Se conservarían domain.py (entidades, Value Objects, máquina de estados, eventos y excepciones) y specifications.py (reglas de descuento), porque no dependen de FastAPI ni de ningún servidor web. Eso es posible porque las reglas viven en el dominio y no en la API.

Conclusión

Con la actividad finalmente se pudo implementar el núcleo de negocio de TechStore para comprobar que las reglas del negocio pueden vivir en el dominio y funcionar sin depender de ninguna interfaz. Los 15 escenarios coinciden con lo esperados y las 39 pruebas de pytest pasan, tanto las del dominio como las de la API. Se encapsulo las reglas en Value Objetc, Specifications y una maquina de estados que evita que una operación invalida deje el pedido en un estado en el que no debería estar. También mostro que registrar los eventos solo después de validar impide anunciar hechos que nunca ocurrieron. Los problemas encontrados, relacionados a las dependencias, la ruta Swagger y la carpeta de ejecución, mostraron que además del código también importa el lugar de su ejecución

Link de Github

https://github.com/valentin-HY/Actividad-4-TechStore